



VeriRAG — Demo Video Narration Script

Track T1 · shares every requirement of the assignment ("What the video must show") · ~7:50 total · **screen recording only, no slides** · every command runs live against the real index — nothing is hard-coded or pre-recorded. Presenter: **SHRESTH**, solo team (every component is explained by the engineer who built it). Rhythm per scene: **point at the screen, say one sentence, type the command, point at the result, say one sentence**. Read the **quote** lines aloud; blue blocks are commands.

QUESTION → RETRIEVE (inverted index) → GENERATE (cited answer) → VERIFY (each citation) → ANSWER

0:00-0:50 | 1. The problem + why it belongs to T1

REQ 1 (≤1 min)

```
python -c "print('QUESTION -> RETRIEVE (inverted index) -> GENERATE (cited answer) -> VERIFY (each citation) -> ANSWER')"
```

```
Get-ChildItem verirag\*.py
```

Say: "This is VeriRAG, for track T1, retrieval-augmented generation and trustworthy answers. The problem: an LLM is only as good as what gets retrieved for it — answers that cannot cite their sources are where hallucinations come from. This project belongs to T1 because the retriever is a real, inspectable IR component, and every generated claim is traced to a ranked source. I'm Shresth, I'm a solo team, and I built every module. The whole package is thirteen modules, about two thousand one hundred lines, pure Python standard library. Corpus: BEIR SciFact, five thousand one hundred eighty-three papers."

0:50-2:05 | 2. The system running end to end on a real query

REQ 2

```
python -m verirag ask "metformin reduces mortality in type 2 diabetes patients"
```

Point at four layers as they print:

- (1) the query line → "that's the question"
- (2) "-- retrieved chunks (k=5) --" → "the retrieve stage, scores on the left"
- (3) "-- generated answer --" → "sentences each ending in [1] [3]"
- (4) "-- citation verification --" + verdicts → "the verify stage"

Say: "One command: retrieve, generate, verify. Every bracket is a real chunk from the index you can open, and every sentence was checked against its chunk before it was shown. These come back supported."

2:05-2:50 | 2b. Same loop in the browser UI (visible end to end)

REQ 2

```
python -m verirag web
```

Say: "Same pipeline, one page, so you can watch the whole loop at once."

Ask tab → paste the same metformin question → Run.

Point at: question box → evidence chunks (yellow = your words) → cited answer with badges → green SUPPORTED strip.

Say: "Question, evidence, cited answer, verdict — the full RAG sequence on screen. Keep this open."

2:50-3:55 | 2c. A limitation, live + 3a. the verify stage's weights & scores

REQ 2 + REQ 3

```
python -m verirag verify "probiotics prevent antibiotic-associated diarrhea"
```

Point at the intermediate output:

the cited chunk → the printed novel terms → the support score with the novelty penalty visible → the verdict: WEAK.

Say: "This is a claim that reads like it might be true — but the verifier scores each sentence against its cited chunk: cosine times one minus zero-point-six times novelty. The words it needs never appear in the chunk, the penalty kicks in, and the verdict is weak. That's the system finding its own limitation, live."

```
python -m verirag ask "probiotics prevent antibiotic-associated diarrhea"
```

Say: "And in the full pipeline the answer is still generated — but the weak sentence is flagged in the verification table. Generation can cite a chunk; the verifier can veto it. That veto is force-on for every answer."

3:55-4:50 | 3b. The pipeline's engine: index + the actual code

REQ 3

```
python -m verirag term resist
```

Point at the printed index internals:

document frequency → idf weight → the postings list → skip pointers.

Say: "Behind 'retrieve' is an inverted index I wrote from scratch — dictionary and postings, idf weights, word positions, skip pointers."

```
Get-Content verirag\index.py -TotalCount 28
```

Say: "And here's the code behind it — the dictionary-and-postings build, zone weights, and the chunking decision: the 'what is a document?' choice. Corpus documents are split into sentence-level chunks that are scored and cited individually, which is what makes span-level citations possible."

4:50-5:40 | 3c. Query processing + scoring (ranked, Boolean, phrase)

REQ 3

```
python -m verirag search "soil bacteria genes"
```

Say: "Ranked cosine search — the words I typed light up in red inside the matched text, and the scores on the left are the cosine scores."

```
python -m verirag bool '"antibiotic resistance" AND bacteria'
```

Say: "The same index answers Boolean and phrase queries — and every step is traced: df-ordered AND, skip-pointer intersection, positional phrase check. Seven documents match."

Say: "Ranked and Boolean retrieval share one index, and every query leaves an audit trail of intermediate postings and scores — not just a top-k screen."

5:40-6:45 | 4. Evaluation: P@k and recall against a baseline

REQ 4

```
python -m verirag eval-verify --charts
```

Point at the table as it prints:

baseline: Jaccard overlap → baseline: cosine only → VeriRAG (cosine – novelty penalty)
– for each: P(SUPPORT), R(SUPPORT), F1, accuracy.

Say: "The verifier, on two hundred held-out claims: Jaccard scores 0.512, plain cosine 0.579, and the novelty-penalized support reaches 0.664 F1, accuracy up from 43 to 67 percent, evidence-sentence recall from 0.10 to 0.57."

```
python -u -m verirag eval --charts
```

Say: "And the retrieval evaluation — every row reports P@5, P@10, recall at 100, nDCG@10 and MRR against the BM25 baseline. The ladder: baseline 0.601, stemming to 0.652, zones 0.656, proximity 0.655 while raising MRR — versus BM25 at 0.653. Champions mode answers under six milliseconds per query. Those are the numbers in the report, reproduced live."

Open output\eval_retrieval.png and output\eval_verifier.png briefly.

Say: "Component ownership — as the solo engineer I present each component I own: the tokenizer and chunker, the document-segmentation decision — mine; the inverted index, zones, scoring and champions lists — mine; the retrieval pipeline and the extractive cited generator — mine; the verifier with its novelty penalty — mine; the evaluation harness — mine. Every component you saw was explained by the person who built it."

Say: "Honest trade-offs: sentence-level chunks cost about two nDCG points versus whole-document indexing, but they buy span-level citations. A static-quality term didn't help on SciFact, and we report that null. LLM generation is wired in, but every number you've seen is the deterministic extractive path. The full four-page report and the README reproduce every number with one command."

Get-ChildItem report

End on one more: `python -m verirag ask "<claim>"` — the LAST image is the RAG loop again.

Coverage — every "What the video must show" item

- ✓ **1. Problem + why it belongs to track T1** — 0:00–0:50 (≤ 1 min)
- ✓ **2. System running end to end on real inputs, incl. a limitation** — 0:50–2:05 (ask), 2:05–2:50 (browser), limitation shown live 2:50–3:55
- ✓ **3. Pipeline, then evaluation; actual code & intermediate output** — pipeline diagram header; index/code 3:55–4:50; postings/weights/scores 2:50–3:55 and 4:50–5:40; evaluation 5:40–6:45
- ✓ **4. Evaluation results against a baseline** — 5:40–6:45 (P@5/P@10/R@100/nDCG@10/MRR vs BM25; verifier F1 vs baselines)
- ✓ **5. Each member explaining the component they own** — 6:45–7:50 (solo: every component presented by its builder)

PowerShell notes: keep the backslash-quotes in the bool command exactly as shown ('\"**antibiotic resistance**\" **AND** **bacteria**'). Run everything from the repo root so Get-ChildItem / Get-Content paths resolve. Between takes, live tinkering: `python -m verirag repl` or `python -m verirag web`. Delivery: upload as **unlisted** (YouTube/Drive), paste the link into README.md *and* the report header; the report also carries the work-division and AI-use-declaration sections.